开源软件基础

单元测试

Zhilei Ren

OSCAR Team, School of Software, Dalian University of Technology

December 4, 2023

Outline

软件测试

软件测试（英语：software testing），描述一种用来促进鉴定软件的正确性、完整性、安全性和质量的过程。据此，您可能会想，软件测试永远不可能完整的确立任意电脑软件的正确性。然而，在可计算理论（计算机科学的一个支派）一个简单的数学证明推断出下列结果：不可能完全解决所谓“死机”，指任意计算机程序是否会进入死循环，或者罢工并产生输出问题。换句话说，软件测试是一种实际输出与预期输出间的审核或者比较过程。
软件测试的经典定义是：在规定的条件下对程序进行操作，以发现程序错误，衡量软件质量，并对其是否能满足设计要求进行评估的过程。[1]

[1] https://zh.wikipedia.org/zh-cn/

今天就能开始用的小知识

哥德尔不完备定理[a]

[a] https://www.bilibili.com/video/BV1fQ4y1Y7qT

任何相容的形式系统，只要蕴涵皮亚诺算术公理，就可以在其中构造在体系中不能被证明的真命题，因此通过推演不能得到所有真命题（即体系是不完备的）。

皮亚诺公理

皮亚诺的五条公理

- 1 是自然数；
- 每一个确定的自然数 a，都有一个确定的后继数 a’，a’ 也是自然数 (一个数的后继数就是紧接在这个数后面的数，例如，1 的后继数是 2，2 的后继数是 3 等等)；
- 对于每个自然数 b、c，b=c 当且仅当 b 的后继数 =c 的后继数；
- 1 不是任何自然数的后继数；
- 任意关于自然数的命题，如果证明了它对自然数 1 是对的，又假定它对自然数 n 为真时，可以证明它对 n’ 也真，那么，命题对所有自然数都真。(这条公理保证了数学归纳法的正确性)

测试的阶段

单元测试

单元测试是对软件组成单元进行测试，其目的是检验软件基本组成单位的正确性，测试的对象是软件设计的最小单位：函数。

测试的阶段

集成测试

集成测试也称综合测试、组装测试、联合测试，将程序模块采用适当的集成策略组装起来，对系统的接口及集成后的功能进行正确性检测的测试工作。其主要目的是检查软件单位之间的接口是否正确，集成测试的对象是已经经过单元测试的模块。

测试的阶段

系统测试

系统测试主要包括功能测试、界面测试、可靠性测试、易用性测试、性能测试。功能测试主要针对包括功能可用性、功能实现程度（功能流程 & 业务流程、数据处理 & 业务数据处理）方面测试。

测试的阶段

回归测试

回归测试指在软件维护阶段，为了检测代码修改而引入的错误所进行的测试活动。回归测试是软件维护阶段的重要工作，有研究表明，回归测试带来的耗费占软件生命周期的 1/3 总费用以上。
与普通的测试不同，在回归测试过程开始的时候，测试者有一个完整的测试用例集可供使用，因此，如何根据代码的修改情况对已有测试用例集进行有效的复用是回归测试研究的重要方向，此外，回归测试的研究方向还涉及自动化工具，面向对象回归测试，测试用例优先级，回归测试用例补充生成等。

单元测试

在计算机编程中，单元测试（英语：Unit Testing）又称为模块测试, 是针对程序模块（软件设计的最小单位）来进行正确性检验的测试工作。程序单元是应用的最小可测试部件。在过程化编程中，一个单元就是单个程序、函数、过程等；对于面向对象编程，最小单元就是方法，包括基类（超类）、抽象类、或者派生类（子类）中的方法。
每个理想的测试案例独立于其它案例；为测试时隔离模块，经常使用 stubs、mock 或 fake 等测试马甲程序。单元测试通常由软件开发人员编写，用于确保他们所写的代码匹配软件需求和遵循开发目标。它的实施方式可以是非常手动的，或者是做成构建自动化的一部分。[2]

[2] https://zh.wikipedia.org/zh-cn/

测试预言

测试准则，又称测试预言（test oracle）是软件测试员或软件工程师用来检测测试是否通过的一种机制。测试准则决定在给定的测试用例输入下产品应有的输出，从而与被测试系统的输出做比较。
常见的测试准则包括：

- 设计规格和软件文档
- 其它产品（例如：作为一个软件程序的测试准则，有可能是使用不同算法计算同一个数学表达式的其它程序）
- 为一组少量测试输入提供近似或准确结果的" 启发式准则"
- 使用统计学特征的" 统计式准则"
- 由使用相同模型而产生和确认系统行为的" 基于模型的准则"

测试预言


晚上11:22
直播
学习
关注
推荐
小韩你们工地最近怎么停工了?
那天大家闲的没事干，把各自的卷尺拿出来比了比
拼多多买的
淘宝买的
直播买的
五金店买的
路上捡的
2002
104
好家伙，我直呼好家伙
不说了，我准备提桶跑路了
129
@建工学霸
提桶跑路才是工地人最后的归宿#工地 #卷尺 #工程人 #建筑工地
@古月法师创作的原声一;
首页
朋友
消息
我

unittest — Unit testing framework

The unittest unit testing framework was originally inspired by JUnit and has a similar flavor as major unit testing frameworks in other languages. It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

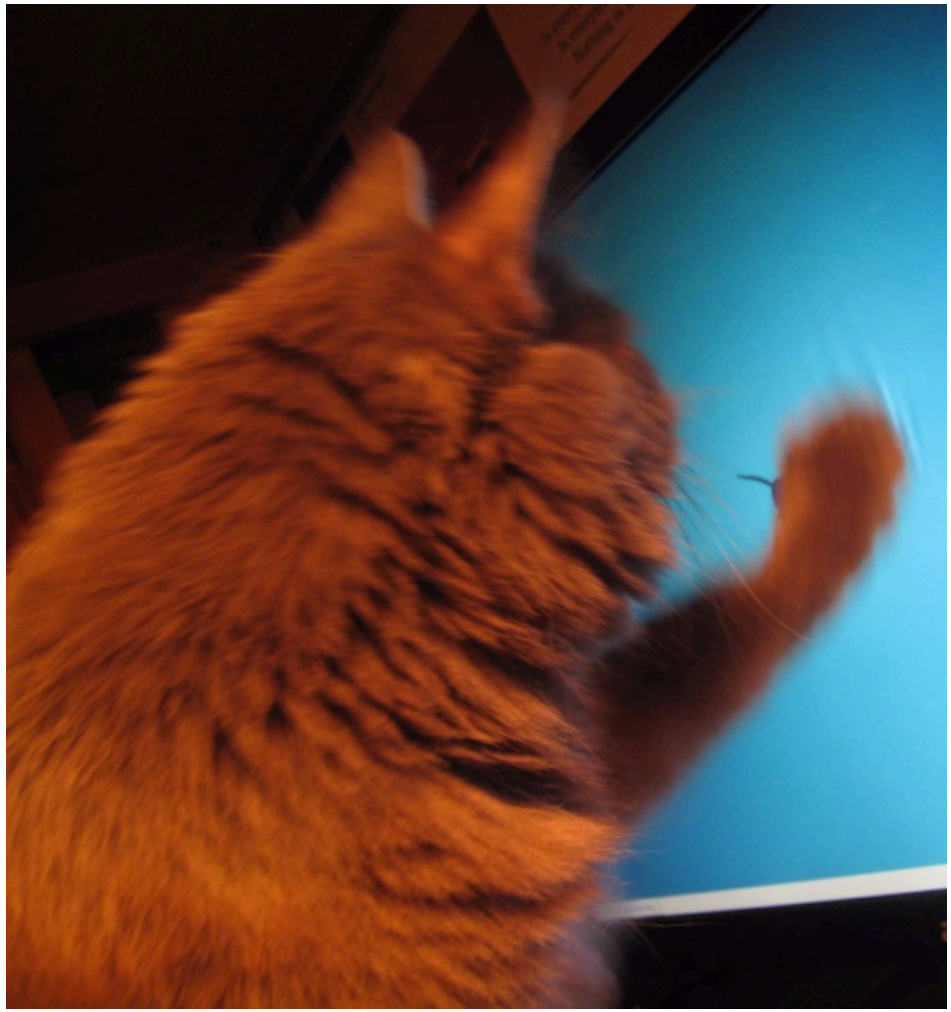
今天就能开始用的小知识

Xfce Bug #12117

The defualt wallpaper is having my animal scritch all the plastic off my LED MONITOR! Can we choose a different wallpaper? I cannot expect the scratches and whu not? Let's end the mouse games over here.[a]

[a] https://bugzilla.xfce.org/show_bug.cgi?id=12117

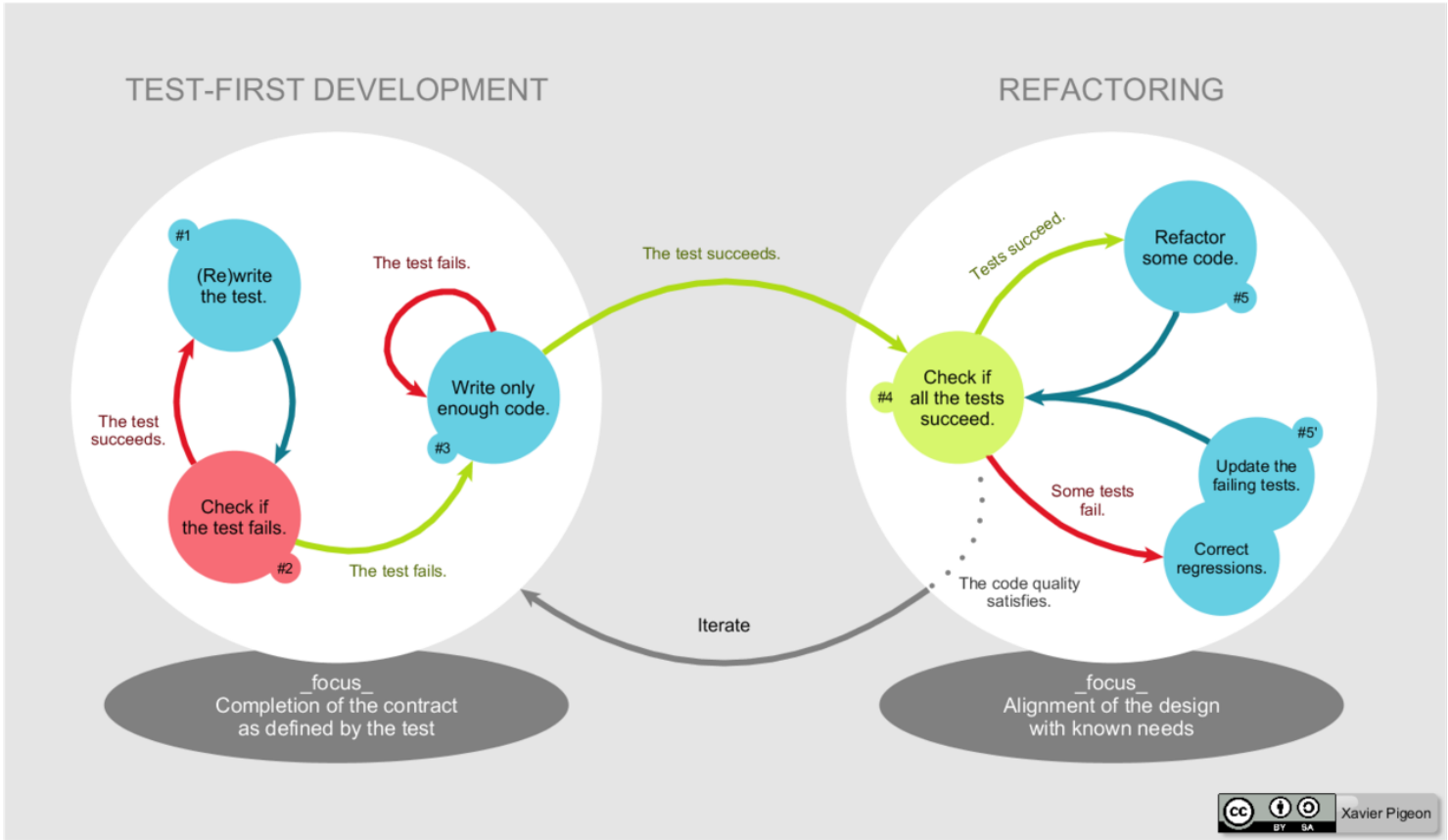
今天就能开始用的小知识



今天就能开始用的小知识

Outline

测试驱动开发[3]



[3] https://en.wikipedia.org/wiki/Test-driven_development

Add a test

In test-driven development, each new feature begins with writing a test. Write a test that defines a function or improvements of a function, which should be very succinct. To write a test, the developer must clearly understand the feature's specification and requirements. The developer can accomplish this through use cases and user stories to cover the requirements and exception conditions, and can write the test in whatever testing framework is appropriate to the software environment. It could be a modified version of an existing test. This is a differentiating feature of test-driven development versus writing unit tests after the code is written: it makes the developer focus on the requirements before writing the code, a subtle but important difference.

Run all tests and see if the new test fails

This validates that the test harness is working correctly, shows that the new test does not pass without requiring new code because the required behavior already exists, and it rules out the possibility that the new test is flawed and will always pass. The new test should fail for the expected reason. This step increases the developer's confidence in the new test.

Write the code

The next step is to write some code that causes the test to pass. The new code written at this stage is not perfect and may, for example, pass the test in an inelegant way. That is acceptable because it will be improved and honed in Step 5.
At this point, the only purpose of the written code is to pass the test. The programmer must not write code that is beyond the functionality that the test checks.

Run tests

If all test cases now pass, the programmer can be confident that the new code meets the test requirements, and does not break or degrade any existing features. If they do not, the new code must be adjusted until they do.

Refactor code

The growing code base must be cleaned up regularly during test-driven development. New code can be moved from where it was convenient for passing a test to where it more logically belongs. Duplication must be removed. Object, class, module, variable and method names should clearly represent their current purpose and use, as extra functionality is added. As features are added, method bodies can get longer and other objects larger. They benefit from being split and their parts carefully named to improve readability and maintainability, which will be increasingly valuable later in the software lifecycle.

Repeat

Starting with another new test, the cycle is then repeated to push forward the functionality. The size of the steps should always be small, with as few as 1 to 10 edits between each test run. If new code does not rapidly satisfy a new test, or other tests fail unexpectedly, the programmer should undo or revert in preference to excessive debugging. Continuous integration helps by providing revertible checkpoints. When using external libraries it is important not to make increments that are so small as to be effectively merely testing the library itself,[4] unless there is some reason to believe that the library is buggy or is not sufficiently feature-complete to serve all the needs of the software under development.

Outline

Basic example

```
import unittest

class TestStringMethods(unittest.TestCase):
    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')
    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())
    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a str
        with self.assertRaises(TypeError):
            s.split(2)
if __name__ == '__main__':
    unittest.main()
```

Basic example

- 测试用例由 unittest.TestCase 子类的实例
- 每个测试函数均由 test 几个字符开始
- 使用 TestCase 父类的 assert* 方法来进行判断
- 使用 setUp 和 tearDown 方法来初始化和结束（类似构造和析构）
- 使用 unittest.main() 调用测试用例
- 使用-v 参数输出详细信息
- 执行顺序按测试函数字母顺序执行

Basic example

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
```

Basic example

```
test_isupper (__main__.TestStringMethods) ... ok
test_split (__main__.TestStringMethods) ... ok
test_upper (__main__.TestStringMethods) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

Assertions

方法	检测条件
assertAlmostEqual(a, b)	round(a-b, 7) == 0
assertNotAlmostEqual(a, b)	round(a-b, 7) != 0
assertGreater(a, b)	a > b
assertGreaterEqual(a, b)	a >= b
assertLess(a, b)	a < b
assertLessEqual(a, b)	a <= b
assertRegex(s, r)	r.search(s)
assertNotRegex(s, r)	not r.search(s)

Command-Line Interface

- 除了在脚本中调用，也可以在命令行调用
- 调用某个文件中的测试用例时，省去文件后缀.py，路径分割符改为"."（类似 Java）
- 同样可以使用-v 输出详细信息
- 不加参数直接使用 python -m unittest 时，开启 discovery 模式

Command-Line Interface

```
python -m unittest test_module1 test_module2
python -m unittest test_module.TestClass
python -m unittest test_module.TestClass.test_method
```

Command-Line Interface

```
python -m unittest -v test_module
```

Command-Line Interface

```
python -m unittest

python -m unittest -h
```

Organizing test code

```
import unittest

class DefaultWidgetSizeTestCase(unittest.TestCase):
    def test_default_widget_size(self):
        widget = Widget('The widget')
        self.assertEqual(widget.size(), (50, 50))
```

Organizing test code

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def test_default_widget_size(self):
        self.assertEqual(self.widget.size(), (50,50),
                         'incorrect default size')

    def test_widget_resize(self):
        self.widget.resize(100,150)
        self.assertEqual(self.widget.size(), (100,150)
                         'wrong size after resize')
```

Organizing test code

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def tearDown(self):
        self.widget.dispose()
```

Outline

测试程序

```
def foo(a, b, x):
    if a > 1 and b == 0:
        x = x / a
    else:
        x = x * a
    if a == 2 or x > 1:
        x = x + 1

if __name__ == "__main__":
    foo(0, 0, 4)
^^I^^I
```

coverage 用法

- coverage run --branch foo.py
- coverage html

coverage 结果

Coverage for **foo.py** : 64%

8 statements | 6 run | 2 missing | 0 excluded | 3 partial

```
1  #!/usr/bin/python3
2
3  def foo(a, b, x):
4      if a > 1 and b == 0:
5          x = x / a
6      else:
7          x = x * a
8      if a == 2 or x > 1:
9          x = x + 1
10
11 if __name__ == "__main__":
12     foo(0, 0, 4)
```